

GABARITO - DESAFIO MÃO NA MASSA

Este é o gabarito completo e comentado do desafio **Mão na Massa**, estruturado para que você possa validar a lógica técnica e a aplicação dos conceitos de encapsulamento, visibilidade e métodos mágicos.

1. Arquivo: Produto.class.php

Este arquivo contém a definição da classe com a lógica de "blindagem" (encapsulamento) e as regras de negócio nos Setters.

```
PHP
<?php

class Produto {
    // Passo 1: Atributos privados (Ninguém acessa de fora da classe)
    private $descricao;
    private $preco;
    private $estoque;

    // Passo 2: Setters com Validação (O "Porteiro" da classe)
    public function setDescricao($valor) {
        // Limpa espaços e converte para maiúsculas antes de salvar
        $this->descricao = strtoupper(trim($valor));
    }

    public function setPreco($valor) {
        // Regra de negócio: Preço deve ser número e maior que zero
        if (is_numeric($valor) && $valor > 0) {
            $this->preco = $valor;
        } else {
            // Dica: Em sistemas reais, poderíamos lançar uma Exception aqui
            error_log("Tentativa de definir preço inválido: $valor");
        }
    }

    public function setEstoque($quantidade) {
        // Regra de negócio: Estoque não pode ser negativo
        if (is_integer($quantidade) && $quantidade >= 0) {
            $this->estoque = $quantidade;
        }
    }

    // Passo 3: Getters (Acesso de leitura controlado)
    public function getDescricao() {
        return $this->descricao;
    }

    public function getPreco() {
        return $this->preco;
    }

    public function getEstoque() {
        return $this->estoque;
    }
}
```

```

    }

    // Passo 4: Método Mágico (Representação em JSON para integração)
    public function __toString() {
        return json_encode([
            "item" => $this->descricao,
            "valor" => $this->preco,
            "estoque" => $this->estoque,
            "data_consulta" => date("Y-m-d H:i:s")
        ]);
    }
}
}

```

2. Arquivo: index.php

Este arquivo serve para testar a robustez da classe (Teste de Stress).

```

PHP
<?php
// Importa a classe
require_once 'Produto.class.php';

echo "<h2>Teste de Robustez - Classe Produto</h2>";

// Instanciando o objeto
$p = new Produto();

// Teste 1: Entrada válida
$p->setDescricao(" monitor led 24 polegadas "); // Deve limpar espaços e ficar MAIÚSCULO
$p->setPreco(850.90);
$p->setEstoque(15);

echo "<b>Teste 1 (Dados Válidos):</b><br>";
echo "Objeto impresso (JSON): " . $p . "<br><br>";

// Teste 2: Entrada inválida (Tentativa de burlar as regras)
echo "<b>Teste 2 (Tentativa de dados inválidos):</b><br>";
$p->setPreco(-100); // O Setter deve bloquear o preço negativo
$p->setEstoque("muitos"); // O Setter deve bloquear pois não é inteiro

echo "Descrição após limpeza: " . $p->getDescricao() . "<br>";
echo "Preço (deve continuar 850.90): " . $p->getPreco() . "<br>";
echo "Estoque (deve continuar 15): " . $p->getEstoque() . "<br><br>";

// Teste 3: Verificando a Blindagem
// echo $p->preco;
// Se você descomentar a linha acima, o PHP gerará um Fatal Error.
// Isso prova que o encapsulamento está funcionando!

echo "<b>Resultado Final:</b><br>";
echo $p; // Chama o __toString() automaticamente

```

3. Análise do Resultado (Checklist Técnico)

Ao executar o `index.php`, VOCÊ deve observar:

1. **Sanitização:** A descrição "monitor led..." apareceu como "MONITOR LED..." (prova que o `setDescricao` funcionou).
2. **Proteção de Dados:** O preço **não mudou** para -100. O valor interno permaneceu o último valor válido (850.90).
3. **Saída de Dados:** Ao usar `echo $p`, o navegador exibiu uma string formatada em JSON, pronta para ser consumida por um frontend em JavaScript ou uma API REST.
4. **Segurança:** Tentar acessar `$p->preco` diretamente resultou em erro, confirmando que a "visibilidade privada" está protegendo os dados sensíveis do sistema.